

Relatório do Projeto Prático
SCC0633 Processamento de Linguagem Natural
SCC5908 Introdução ao Processamento de Língua Natural

1 Nome do projeto

PLN Core: analisador simbólico de sentimentos para português brasileiro

2 Equipe

Nome da equipe: GLAVJ

- João Victor M. Correa (17416061)
- Ana Paula de Abreu Batista (12688424)
- Guilherme Vaz de Oliveira Taratá (10817476)
- Victoria Favero Nunes (15698302)
- Leonardo Ferreira Runho (11832958)

2.1 Autoria do grupo e recursos externos

A Tabela 1 separa o que foi implementado pela equipe do que foi reaproveitado de recursos externos. Todo o pipeline simbólico, a normalização, as regras, o módulo de recomendação, o catálogo de músicas e os testes automatizados são autoria do grupo. Os recursos externos limitam-se ao léxico OpLexicon v3.0 e ao modelo spaCy `pt_core_news_sm`.

Feito pelo grupo	Recurso externo
Pipeline simbólico (<code>pipeline.py</code>)	Léxico OpLexicon v3.0 (Souza et al., 2011)
Normalização e <i>folding</i> (<code>text_utils.py</code>)	spaCy <code>pt_core_news_sm</code> (Explosion AI, 2026)
Regras de negação, intensificação, atenuação, contraste e ênfase	—
Limiares de classificação e agregação	—
Módulo de recomendação (<code>recommender.py</code>) e catálogo de 10 músicas (<code>recommendations.json</code>)	—
Conjunto de 24 testes unitários (<code>tests/</code>) e <i>script</i> de avaliação (<code>scripts/evaluate.py</code>)	—

Tabela 1: Separação entre autoria do grupo e recursos externos reaproveitados.

3 Introdução

Este projeto tem como objetivo classificar textos curtos em português brasileiro como positivos, negativos ou neutros. A entrada consiste em frases simples, como comentários de redes sociais, e a saída é um rótulo acompanhado de um escore numérico.

A análise de sentimentos é uma tarefa amplamente utilizada em aplicações como monitoramento de opinião pública, avaliação de produtos e apoio à tomada de decisão. Em ambientes digitais, há um grande volume de opiniões textuais produzidas continuamente, o que torna essa tarefa relevante tanto do ponto de vista prático quanto acadêmico.

No contexto do português brasileiro, essa tarefa apresenta desafios adicionais, como variações linguísticas, uso de gírias, ambiguidade semântica e forte dependência de contexto. Entre os principais desafios estão:

- **Dependência de contexto** (ironia e sarcasmo): em “adorei ficar duas horas esperando no aeroporto”, todas as palavras polarizadas são positivas, mas a intenção é negativa.
- **Ambiguidade lexical**: uma mesma palavra carrega polaridades distintas conforme o contexto (Navigli, 2009). Em português brasileiro, “duro” pode ser físico (neutro), financeiro (“estou duro”, negativo) ou de personalidade (negativo); “bomba” pode ser elogio em academia (“bombou no treino”) ou crítica em outro contexto.
- **Negações e intensificadores**: alteram ou amplificam a polaridade dentro de uma janela curta de tokens, como em “não foi bom” (inversão) ou “muito bom” (ampliação).
- **Cobertura limitada de léxicos**: gírias frequentes em redes sociais (“top”, “mara”, “show de bola”, “irado”) raramente aparecem em léxicos tradicionais, mesmo nos mais amplos como o OpLexicon v3.0 (31 944 termos).

Além disso, trata-se de um problema didaticamente relevante em Processamento de Língua Natural, pois envolve a modelagem de fenômenos linguísticos complexos de forma estruturada. Neste trabalho, adotamos uma abordagem simbólica baseada em léxico e regras explícitas, priorizando interpretabilidade em vez de aprendizado automático (Taboada et al., 2011; Hutto and Gilbert, 2014; Souza and Vieira, 2012).

4 Proposta da solução

4.1 Arquitetura do pipeline

A aplicação segue um pipeline simbólico determinístico. Diferente de modelos de aprendizado de máquina, não há treinamento: o processamento ocorre por etapas fixas e interpretáveis.

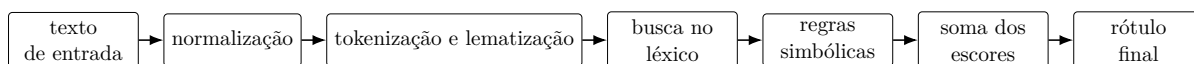


Figura 1: Visão geral da arquitetura da solução.

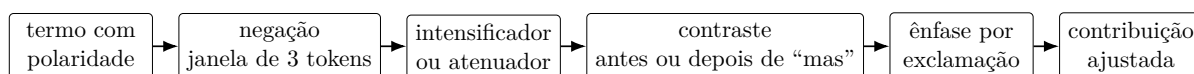


Figura 2: Fluxo local de ajuste para cada termo encontrado no léxico.

4.2 Recursos e ferramentas

A Tabela 2 detalha os três recursos do sistema: a função, a origem (própria do grupo ou externa), o detalhamento técnico e a versão.

Recurso	Função	Detalhamento	Origem
Expressões regulares (<code>text_utils.py</code>)	Normalização do texto bruto	Remove URLs e menções, mantém a palavra após o #, comprime caracteres repetidos (“ameiiii” → “ameii”), normaliza espaços e, em seguida, faz <i>folding</i> (minúsculas e remoção de acentos).	Grupo
spaCy pt_core_news_sm	Tokenização e lematização	Modelo pré-treinado em português que produz lemas (“gostei” → “gostar”, “confusas” → “confuso”). Lemas auxiliares (ser , estar , ter , haver , ir) são filtrados para evitar falsos positivos.	spaCy 3.x (Explosion AI, 2026)
OpLexicon v3.0	Polaridade lexical	Léxico com 31 944 termos distintos após agregar duplicatas pela média. Cada linha traz termo, tipo, polaridade, revisão ; a polaridade é discreta no conjunto $\{-1, 0, +1\}$. Baixado e cacheado automaticamente na primeira execução.	PUCRS (Souza et al., 2011)

Tabela 2: Recursos utilizados pelo sistema, com função, detalhamento e origem.

4.3 Descrição dos processos

Escolha do léxico. O sistema utiliza o OpLexicon v3.0 como recurso externo de polaridade. Os dados são carregados automaticamente e entradas duplicadas são agregadas por média de polaridade, produzindo o *escore base* usado nas etapas seguintes.

Normalização. Consiste na remoção de URLs e menções, normalização de espaços e redução de caracteres repetidos.

Tokenização e Lematização. A CLI e a interface Streamlit usam o modelo `pt_core_news_sm` do spaCy (Explosion AI, 2026). O texto normalizado é tokenizado pelo modelo e cada item é convertido para sua forma lematizada, permitindo associar flexões como **gostei** e **gostaram** à forma **gostar** presente no léxico. Lemas auxiliares muito frequentes, como **ser** e **estar**, são filtrados para evitar correspondências espúrias no OpLexicon.

Busca no léxico. Cada lema localizado no recurso recebe um *escore base* de polaridade, ou seja, o valor anotado para aquele termo antes de qualquer ajuste por regra. No OpLexicon v3.0 esse valor é discreto no conjunto $\{-1, 0, +1\}$; quando o mesmo termo aparece em múltiplas entradas, a média entre elas é usada. Marcadores funcionais (negações e contrastes) são identificados nesta etapa mas não recebem polaridade própria: eles atuam apenas como gatilhos das regras descritas a seguir.

Regras simbólicas. A Tabela 3 resume cada regra com gatilho, janela de aplicação, efeito numérico e exemplo. Os pesos foram calibrados manualmente pela equipe e seguem o espírito de abordagens como o VADER (Hutto and Gilbert, 2014) e a análise de negadores e modais de Kiritchenko and Mohammad (2016).

Agregação e classificação. O escore final da frase é a soma dos escores ajustados de todos os termos casados. A classificação aplica dois limiares simétricos sobre essa soma:

$$\text{score} = \sum_i \text{adjusted}(t_i), \quad \text{rótulo} = \begin{cases} \text{positivo} & \text{se score} \geq +0,75 \\ \text{negativo} & \text{se score} \leq -0,75 \\ \text{neutro} & \text{caso contrário} \end{cases} \quad (1)$$

Regra	Gatilho	Janela	Efeito	Exemplo
Negação	não, nem, nunca, jamais, sem	3 tokens anteriores	Multiplica por -1 (inverte sinal)	“não foi bom” $\rightarrow$ bom $\times(-1)$
Intensificação	muito 1,6; super 1,5; demais 1,4; bastante 1,3; bem/realmente 1,2	token imediatamente anterior	Multiplica pelo fator do gatilho	“muito bom” $\rightarrow$ bom $\times 1,6$
Atenuação	pouco 0,6; meio 0,75; quase 0,8	token imediatamente anterior	Multiplica pelo fator do gatilho	“pouco útil” $\rightarrow$ útil $\times 0,6$
Contraste	mas, porém, contudo, entretanto (último marcador da frase)	toda a frase	Termos antes: $\times 0,7$; depois: $\times 1,3$	“foi bom, mas péssimo”
Exclamação	“!” no texto bruto (até 3)	toda a frase	Multiplica por $1 + 0,05 \cdot n$	“bom!!” $\rightarrow$ bom $\times 1,10$

Tabela 3: Regras simbólicas, com gatilho, janela, efeito numérico e exemplo.

Quando nenhum termo é encontrado no léxico, o rótulo é **neutro** por definição.

4.4 Exemplo de execução

A Tabela 4 ilustra o pipeline completo para a entrada **O app é muito bom, mas a tela confusa!**. Cada contribuição ajustada é o produto do score base pelos fatores das regras aplicadas, conforme a Tabela 3.

Etapas	Resultado
Entrada	O app é muito bom, mas a tela confusa!
Normalização	remoção de ruídos e padronização de espaços; neste caso o texto permanece igual
Tokenização e lematização	o, app, muito, bom, mas, o, tela, confuso
Busca no léxico	bom (+1,0) e confuso (−1,0) encontrados no OpLexicon; mas é tratado como marcador funcional de contraste, sem polaridade própria
Regras aplicadas	bom : intensificação (muito), pré-contraste, exclamação; confuso : pós-contraste, exclamação
Contribuições ajustadas	bom : $1,0 \times 1,6 \times 0,7 \times 1,05 = +1,176$; confuso : $-1,0 \times 1,3 \times 1,05 = -1,365$
Agregação e classificação	soma final $-0,189$; como $ -0,189 < 0,75$, classificada como neutro

Tabela 4: Aplicação completa do pipeline para **O app é muito bom, mas a tela confusa!**.

Esse exemplo mostra como evidências positivas e negativas podem se equilibrar: a palavra positiva inicial (**bom**, com intensificador) é atenuada pelo pré-contraste, enquanto a palavra negativa final (**confuso**) é amplificada pelo pós-contraste. A soma fica próxima de zero e o sistema, prudentemente, devolve **neutro**, em linha com a leitura humana de que a frase oscila entre elogio e crítica.

5 Execução

O repositório remoto do projeto é:

<https://github.com/vicmcorreia/PLN-core>

Para executar o projeto, é necessário preparar o ambiente virtual e instalar as dependências.

Execução via linha de comando

Linux/MacOS:

```
./install.sh
source .venv/bin/activate
python main.py
```

Windows:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools
python -m pip install -e .
python main.py
```

Também é possível analisar diretamente um texto:

```
python main.py "Eu amei o filme, foi muito bom!"
```

O sistema permite dois modos:

- análise de texto individual
- modo de comparação dos exemplos embutidos usando o mesmo pipeline do aplicativo

Recursos adicionais

Na primeira execução, o léxico OpLexicon v3.0 é automaticamente baixado e armazenado localmente.

Além da interface em linha de comando, o projeto também oferece uma interface web simples utilizando Streamlit:

```
streamlit run streamlit_app.py
```

ou, via opção acessível diretamente no navegador:

<https://pln-core.streamlit.app/>

A interface permite visualizar:

- rótulo final
- escore
- texto normalizado
- lemas gerados para busca no léxico
- termos encontrados no léxico
- regras simbólicas aplicadas

Testes

Para executar os testes atuais do projeto, após o ambiente estar preparado:

```
python -m unittest discover -s tests
```

6 Recomendação de músicas

Como extensão do sistema, foi implementado um módulo de recomendação de músicas baseado no sentimento detectado no texto.

Catálogo. O catálogo, descrito em `data/recommendations.json`, é um pequeno conjunto curado pela equipe com 10 músicas brasileiras de domínio reconhecido, distribuídas em 4 positivas, 3 negativas e 3 neutras. Cada entrada traz título, artista, identificador no YouTube, rótulo de sentimento (os mesmos três do classificador) e valor de valência. Os rótulos e as valências foram atribuídos manualmente pelo grupo, considerando letra e clima geral da gravação. Nenhum modelo externo, API ou serviço de *streaming* foi utilizado.

Valência. O conceito vem da psicologia do afeto (Russell, 1980) e é também o nome do atributo de mesma natureza na Spotify Web API. Trata-se de uma medida do grau de positividade emocional de uma faixa: valência alta indica clima alegre e brilhante, valência baixa indica clima triste ou melancólico. Para casar com o escore do classificador, usamos o intervalo $[-1, +1]$. Trata-se de uma aproximação deliberadamente simples, e por isso a rotulagem manual é registrada como limitação na Seção 8.

Regra de recomendação. Dado um texto, o sistema filtra músicas com o mesmo rótulo e as ordena pela menor distância entre o escore do texto e a valência da música:

$$d(s) = |\text{valence}(s) - \text{score}_{\text{texto}}| \quad (2)$$

A música recomendada é aquela com menor $d(s)$. O módulo é determinístico e não utiliza aprendizado de máquina.

Resultados. A Tabela 5 mostra a saída real do sistema para quatro entradas representativas, cada uma processada pelo pipeline completo (OpLexicon v3.0 + spaCy + módulo de recomendação).

Texto	Rótulo	Escore	Música recomendada	Valência
Eu amei o filme, foi muito bom!	positive	+2,73	País Tropical (Jorge Ben Jor)	+0,90
Não gostei do app, está bem confuso e bugado.	negative	-2,20	Cálice (Chico Buarque & Milton Nascimento)	-0,90
O arquivo tem quatro páginas e duas tabelas.	neutral	0,00	Águas de Março (Tom Jobim & Elis Regina)	+0,10
Estou triste com o resultado.	negative	-1,00	Tempo Perdido (Legião Urbana)	-0,55

Tabela 5: Saídas reais do módulo de recomendação para quatro entradas representativas.

7 Resultados

Testes unitários. O projeto inclui 24 testes automatizados em `tests/`, cobrindo o pipeline simbólico, o carregamento do léxico, os tokenizadores, o módulo de recomendação e a CLI. Todos passam (Ran 24 tests in 0.27s OK).

Avaliação qualitativa. Para uma medida acima do nível de testes unitários, escrevemos o *script* `scripts/evaluate.py` com 20 sentenças curtas anotadas manualmente em três classes (8 positivas, 8 negativas, 4 neutras). A Tabela 6 compara o pipeline de produção (OpLexicon + spaCy) com o pipeline didático que usa o nosso léxico-semente, mantido como controle.

Configuração	Acertos	Acurácia
Seed lexicon (controle didático)	19/20	95%
OpLexicon v3.0 + spaCy <code>pt_core_news_sm</code> (produção)	14/20	70%

Tabela 6: Acurácia em 20 sentenças curtas balanceadas, executadas pelo *script* `scripts/evaluate.py`.

Análise dos erros. Os 6 erros do pipeline de produção concentram-se em três padrões: (i) gírias frequentes em português brasileiro que não estão no OpLexicon (“top”, “mara”), produzindo falsos neutros; (ii) substantivos comuns que recebem polaridade no léxico, gerando falsos positivos em frases neutras (“produto”, “pedido”); e (iii) palavras com polaridade pequena que ficam abaixo do limiar de 0,75 mesmo com regras aplicadas. Esses padrões dão pistas claras para evoluções futuras: ampliação do léxico com gírias, depuração de substantivos de alto uso e ajuste fino dos limiares por classe.

8 Conclusão

O presente trabalho apresentou um sistema simbólico para análise de sentimentos em português brasileiro, baseado em léxico e regras explícitas, priorizando interpretabilidade ao longo de todo o pipeline.

Os números da Seção 5 mostram que a abordagem captura padrões claros de polaridade em sentenças curtas (70% de acurácia no pipeline de produção e 95% no léxico-semente didático), mas também evidenciam limitações inerentes: a dependência da cobertura lexical, a simplificação de fenômenos linguísticos complexos por meio de regras heurísticas e a análise predominantemente local do contexto impactam diretamente a qualidade das classificações.

Além disso, o módulo de recomendação, embora funcional, baseia-se em valores de valência atribuídos manualmente pela equipe, o que limita a precisão das sugestões fora do catálogo curado.

Essas limitações são esperadas em abordagens simbólicas e apontam caminhos naturais para evolução do sistema, como o enriquecimento do léxico com gírias e termos de domínio, o refinamento das regras com base em análise sistemática dos erros e a incorporação de técnicas mais robustas de modelagem linguística.

Em síntese, o projeto evidencia que, embora abordagens simbólicas ofereçam transparência e controle, sua efetividade depende fortemente da qualidade dos recursos e das simplificações adotadas, reforçando o papel dessas técnicas como um ponto de partida interpretável, mas não definitivo, para a análise de sentimentos.

Referências

- Explosion AI. Models and languages, 2026. URL <https://spacy.io/docs/usage/models>. spaCy usage documentation accessed in April 2026.
- C. J. Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the International AAAI Conference on Web and Social Media*, 2014. URL <https://ojs.aaai.org/index.php/ICWSM/article/view/14550>.

- Svetlana Kiritchenko and Saif M. Mohammad. The effect of negators, modals, and degree adverbs on sentiment composition. In *Proceedings of the Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*, 2016. URL <https://aclanthology.org/W16-0410/>.
- Roberto Navigli. Word sense disambiguation: A survey. *ACM Computing Surveys*, 41(2), 2009. doi: 10.1145/1459352.1459355. URL <https://doi.org/10.1145/1459352.1459355>.
- James A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178, 1980. doi: 10.1037/h0077714. URL <https://doi.org/10.1037/h0077714>.
- Marlo Souza and Renata Vieira. Sentiment analysis on twitter data for portuguese language. In *Computational Processing of the Portuguese Language*, 2012. doi: 10.1007/978-3-642-28885-2_28. URL https://doi.org/10.1007/978-3-642-28885-2_28.
- Marlo Souza, Renata Vieira, Debora Buseti, Rove Chishman, and Isa Mara Alves. Construction of a portuguese opinion lexicon from multiple resources. In *Proceedings of the 8th Brazilian Symposium in Information and Human Language Technology*, 2011. URL <https://aclanthology.org/W11-4507/>.
- Maite Taboada, Julian Brooke, Milan Tofiloski, Kimberly Voll, and Manfred Stede. Lexicon-based methods for sentiment analysis. *Computational Linguistics*, 2011. URL <https://aclanthology.org/J11-2001/>.